Here is the **version-wise SOLID principle implementation analysis of java.lang.Object from Java 1.0 to Java 21**.

 Important:
Object has remained structurally unchanged since Java 1.0.
Therefore, SOLID implementation is **conceptual and design-based**, not version-dependent.
Only finalize() became deprecated starting Java 9.

 SOLID Principles in java.lang.Object (Java 1 → 21)

SOLID Principle	How Object Implements It	Java 1–8	Java 9–21	Notes
S — Single Responsibility Principle (SRP)	Provides only fundamental behaviors common to all objects (identity, equality, synchronization, representation).	✅	✅	Core minimal responsibilities maintained across all versions
O — Open/Closed Principle (OCP)	Methods like equals(), hashCode(), toString(), clone() are designed to be overridden without modifying Object itself.	✅	✅	Stable since Java 1.0
L — Liskov Substitution Principle (LSP)	Every subclass must remain substitutable for Object. All objects can be treated polymorphically as Object.	✅	✅	Root of Java inheritance hierarchy
I — Interface Segregation Principle (ISP)	Object does not force subclasses to implement unnecessary methods (no abstract methods).	✅	✅	No mandatory contract methods
D — Dependency Inversion Principle (DIP)	High-level code depends on Object abstraction instead of concrete classes (e.g., Collections use Object).	✅	✅	Foundation for polymorphism

---

□ Detailed Breakdown Per Principle

Principle	Supporting Methods in Object	Design Behavior
SRP	All 11 methods	Only universal object-level responsibilities
OCP	equals(), hashCode(), toString(), clone()	Designed for overriding
LSP	Entire class	Every class extends Object
ISP	No abstract methods	No forced implementation
DIP	Used as base type everywhere	Enables loose coupling

⚠ Version-Specific Note (Java 9+)

Change	Impact on SOLID
finalize() deprecated	Improves SRP and OCP by removing problematic cleanup responsibility

---

□ Stability Summary (Java 1 → 21)

Aspect	Status
Methods Added	✕ None
Methods Removed	✕ None
Methods Deprecated	finalize() (Java 9+)

Structural Changes	❌ None
SOLID Alignment	Consistent across all versions

□ Why Object Strongly Aligns with SOLID

1. Minimal responsibilities
2. Designed for safe overriding
3. Supports polymorphism
4. Does not enforce contracts
5. Encourages abstraction-first design

□ Final Conclusion

From **Java 1.0 (1996)** to **Java 21 (2023)**:

- Object fully aligns with **all five SOLID principles**
- No structural change was required
- It remains one of the most stable foundational designs in software engineering

---

If you want, I can next provide:

- ✔UML-style SOLID mapping diagram
- ✔SOLID comparison: Object vs String
- ✔Bytecode-level SOLID explanation
- ✔JVM architecture relationship with SOLID principles

Here is a **comprehensive list of software design principles reflected in java.lang.Object from Java 1.0 to Java 21**.

 Note: The structure of Object has remained stable since **Java 1.0**, so these principles apply consistently across all versions (except finalize() being deprecated in Java 9+).

 Design Principles Implemented in java.lang.Object (Java 1 → 21)

#	Principle	How It Is Reflected in Object	Related Methods / Features	Java 1–8	Java 9–21
1	**Single Responsibility Principle (SRP)**	Provides only universal object behaviors (identity, equality, synchronization)	All methods	✅	✅
2	**Open/Closed Principle (OCP)**	Methods designed for overriding without modifying base class	equals(), hashCode(), toString(), clone()	✅	✅
3	**Liskov Substitution Principle (LSP)**	Every class can substitute Object	Entire class hierarchy	✅	✅
4	**Interface Segregation Principle (ISP)**	No abstract methods forcing subclasses	No mandatory contracts	✅	✅
5	**Dependency Inversion Principle (DIP)**	Code depends on Object abstraction	Polymorphism usage	✅	✅
6	**Encapsulation Principle**	Methods define controlled object interaction	All public/protected methods	✅	✅
7	**Polymorphism Principle**	Supports dynamic method	equals(), toString()	✅	✅

		dispatch			
8	**Abstraction Principle**	Provides abstract representation of any entity	Root class design	✓	✓
9	**Immutability Support Principle**	Does not enforce mutability; supports immutable subclasses	Design flexibility	✓	✓
10	**Fail-Fast Design**	wait() throws InterruptedException	Thread methods	✓	✓
11	**Resource Management Principle**	Cleanup support via finalize() (now deprecated)	finalize()	✓	⚠ Deprecated
12	**Thread-Safety / Monitor Pattern**	Built-in monitor lock per object	wait(), notify(), notifyAll()	✓	✓
13	**Contract-Based Design**	Defines equality/hashCode contract	equals() & hashCode()	✓	✓
14	**Minimalism Principle**	Only 11 core methods since 1996	Entire class	✓	✓
15	**Backward Compatibility Principle**	No breaking changes in 25+ years	Entire design	✓	✓
16	**Extensibility Principle**	Subclasses extend behavior safely	Non-final overridable methods	✓	✓
17	**Information Hiding**	Internal object state not exposed directly	Design philosophy	✓	✓

□ Version-Specific Note

Java Version	Notable Design Impact
Java 1.0	Core principles established
Java 9	finalize() deprecated (improves SRP & resource safety)
Java 18+	Deprecated for removal (Cleaner API preferred)
Java 21	Structure unchanged

□ Principle Coverage Summary

Category	Count
SOLID Principles	5
Core OOP Principles	4
Concurrency Principles	2
Design Stability Principles	3
Total Principles Reflected	**17**

□ Key Insight

Object is one of the most stable foundational designs in programming history:

- ✔No new methods added in 25+ years
- ✔No removals
- ✔Strong adherence to OOP and SOLID
- ✔Designed for universal inheritance

If you want next, I can provide:

- ✔UML-style architectural diagram of principles mapping
- ✔JVM internal relationship with Object
- ✔Comparison: Object vs String principles
- ✔Principle impact on Collections Framework

Here is the **UML-style architectural explanation of java.lang.Object (Java 1 → 21)** presented in table form.

 Note: The structure of Object has remained unchanged since Java 1.0 (except finalize() deprecated in Java 9+).

 UML-Style Architectural View of java.lang.Object

UML Element	Representation in Object	Visibility	Type	Description	Design Role
Class	Object	public	Concrete Root Class	Top-most superclass of all Java classes	Root of inheritance hierarchy
Generalization (Inheritance)	AnyClass ⟶ Object	Implicit	IS-A Relationship	Every class extends Object automatically	Enables polymorphism
Operation	+ equals(Object): boolean	public	Instance	Logical equality	Supports overriding

			Method	comparison	
Operation	+ hashCode(): int	public	Instance Method	Hash value generation	Hash-based collections
Operation	+ toString(): String	public	Instance Method	String representation	Debugging/logging
Operation	+ final getClass(): Class<?>	public final	Instance Method	Returns runtime class	Reflection support
Operation	# clone(): Object	protected	Instance Method	Creates copy of object	Prototype behavior
Operation	# finalize(): void ⚠	protected	Instance Method	Cleanup before GC	Deprecated (Java 9+)
Operation	+ final wait(): void	public final	Instance Method	Thread waits indefinitely	Monitor mechanism
Operation	+ final wait(long): void	public final	Instance Method	Timed wait	Thread coordination
Operation	+ final wait(long,int): void	public final	Instance Method	Timed wait (ns precision)	Thread coordination
Operation	+ final notify(): void	public final	Instance Method	Wakes one waiting thread	Monitor signaling
Operation	+ final notifyAll(): void	public final	Instance Method	Wakes all waiting threads	Monitor signaling
Association	Monitor Lock	Internal	JVM Feature	Every object has intrinsic lock	Concurrency foundation

Dependency	Used by Collections, Reflection, Threads	External	Usage Relationship	Core dependency across JDK	Base abstraction

---

□ UML Visibility Legend

Symbol	Meaning
+	Public
#	Protected
final	Cannot be overridden
⚠	Deprecated

□ Architectural Characteristics (Java 1 → 21)

Aspect	Status
Total Methods	11
Added Since Java 1	None
Removed	None
Deprecated	finalize()
Abstract Methods	None
Interfaces Implemented	None

Extensibility	High (via overriding allowed methods)
Stability	Extremely High

□ Simplified UML Structural Diagram (Text Representation)

```
+---------------------------+
|    Object                 |
+---------------------------+
| + equals()                |
| + hashCode()              |
| + toString()              |
| + getClass()              |
| # clone()                 |
| # finalize() ⚠            |
| + wait()                  |
| + notify()                |
+---------------------------+
        ▲
        |
(All Java Classes)
```

□ Architectural Insight

- Object is a **Concrete Root Base Class**
- Provides:
  - Identity behavior
  - Equality contract
  - Runtime type metadata
  - Built-in synchronization
- Designed for:
  - Extensibility

- Polymorphism
- Backward compatibility

If you want next, I can provide:

- ✔UML + SOLID combined architectural mapping
- ✔JVM internal memory diagram of Object
- ✔Bytecode-level architectural explanation
- ✔Comparison UML: Object vs String

Here is a **version-wise comparison of the Object class** from **Java 1.0 to Java 21**.

 Note: java.lang.Object is the root class of all Java classes and has remained **structurally stable since Java 1.0**. No new methods were added from Java 1 to Java 21.

 Java Object Class Evolution (Java 1 → Java 21)

Java Version	Methods Available in Object	Changes Introduced	Notes
Java 1.0 (1996)	equals()hashCode()toString()getClass()clone()finalize()wait()wait(long)wait(long,int)notify()notifyAll()	Initial release	Core object behavior defined
Java 1.1	Same as Java 1.0	No change	Reflection API introduced (affects getClass() usage)
Java 1.2 (Collections Framework)	Same	No change	Collections rely heavily on equals() and hashCode()

Java 1.3	Same	No change	JVM optimizations
Java 1.4	Same	No change	Assertion mechanism introduced
Java 5 (1.5)	Same	No change	Generics introduced (affects casting from Object)
Java 6	Same	No change	Performance improvements
Java 7	Same	No change	Minor JVM updates
Java 8	Same	No change	Lambda & Streams (impact usage, not structure)
Java 9 (Module System)	Same	finalize() deprecated	Marked deprecated due to performance & security issues
Java 10	Same	No structural change	
Java 11 (LTS)	Same	No change	
Java 12–17	Same	No change	
Java 18	Same	finalize() deprecated for removal	Stronger warning
Java 19	Same	No change	
Java 20	Same	No change	
Java 21 (LTS)	Same	finalize() still deprecated	Recommended to use Cleaner API

▯ Complete List of Object Class Methods (Java 21)

Category	Methods
Identity & Comparison	equals(Object obj)
Hashing	hashCode()
String Representation	toString()
Runtime Class Info	getClass()
Cloning	protected Object clone()
Finalization	protected void finalize() ▯ Deprecated
Thread Communication	wait()wait(long timeout)wait(long timeout, int nanos)notify()notifyAll()

▯ Key Observations

Aspect	Status from Java 1 → 21
Number of Methods	11 (unchanged)
Methods Added	✕ None
Methods Removed	✕ None
Deprecated	finalize() (Java 9+)
Design Stability	Extremely Stable

⍰ Why Object Rarely Changes?

- It is the **root of the Java class hierarchy**
- Any change would break backward compatibility
- JVM heavily depends on it
- Core to polymorphism and inheritance

⍰ UML-Style Structural View

```
java.lang.Object
 ├── equals(Object)
 ├── hashCode()
 ├── toString()
 ├── getClass()
 ├── clone()
 ├── finalize()   (Deprecated)
 ├── wait()
 ├── wait(long)
 ├── wait(long,int)
 ├── notify()
 └── notifyAll()
```

⍰ Conclusion

From **Java 1.0 to Java 21**, the Object class:

- ✅ Remained structurally unchanged
- ⍰ Only finalize() became deprecated
- ⍰ Maintains strong backward compatibility

If you want, I can also provide:

- ✔Version-wise JVM behavior changes related to Object
- ✔SOLID principles reflection in Object
- ✔Internal JVM implementation comparison
- ✔Detailed method-level behavioral evolution table

Here is the **complete version-wise comparison of all Object class methods** from **Java 1.0 to Java 21**.

 Important: The java.lang.Object class has had **exactly the same 11 methods since Java 1.0**.
No new methods were ever added. No methods were removed.
Only finalize() was deprecated starting in Java 9.

 Complete Version-wise Table of Object Methods (Java 1 → 21)

Java Version	equals	Hash Code	toString	getClass	clone	finalize	wait()	Wait (long)	Wait (long,int)	notify	Notify All	Changes
Java 1.0	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	Initial Release
Java 1.1	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	No Change
Java 1.2	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	No Change
Java 1.3	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	No Change
Java 1.4	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	No Change
Java 5	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	No Change

(1.5)												
Java 6	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	No Change
Java 7	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	No Change
Java 8	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	No Change
Java 9	✓	✓	✓	✓	✓	⚠ Deprecated	✓	✓	✓	✓	✓	finalize() deprecated
Java 10	✓	✓	✓	✓	✓	⚠ Deprecated	✓	✓	✓	✓	✓	No structural change
Java 11 (LTS)	✓	✓	✓	✓	✓	⚠ Deprecated	✓	✓	✓	✓	✓	No Change
Java 12	✓	✓	✓	✓	✓	⚠ Deprecated	✓	✓	✓	✓	✓	No Change
Java 13	✓	✓	✓	✓	✓	⚠ Deprecated	✓	✓	✓	✓	✓	No Change
Java 14	✓	✓	✓	✓	✓	⚠ Deprecated	✓	✓	✓	✓	✓	No Change
Java 15	✓	✓	✓	✓	✓	⚠ Deprecat	✓	✓	✓	✓	✓	No Change

						ed						
Java 16	✓	✓	✓	✓	✓	⚠ Deprecated	✓	✓	✓	✓	✓	No Change
Java 17 (LTS)	✓	✓	✓	✓	✓	⚠ Deprecated	✓	✓	✓	✓	✓	No Change
Java 18	✓	✓	✓	✓	✓	⚠ Deprecated for removal	✓	✓	✓	✓	✓	Stronger warning
Java 19	✓	✓	✓	✓	✓	⚠ Deprecated for removal	✓	✓	✓	✓	✓	No Change
Java 20	✓	✓	✓	✓	✓	⚠ Deprecated for removal	✓	✓	✓	✓	✓	No Change
Java 21 (LTS)	✓	✓	✓	✓	✓	⚠ Deprecated for removal	✓	✓	✓	✓	✓	No Change

---

▯ Complete Method Signatures (Java 21)

Method	**Signature**	**Category**

equals	public boolean equals(Object obj)	Equality
hashCode	public int hashCode()	Hashing
toString	public String toString()	String Representation
getClass	public final Class<?> getClass()	Runtime Type Info
clone	protected Object clone() throws CloneNotSupportedException	Object Copy
finalize	protected void finalize() throws Throwable	Cleanup (Deprecated)
wait	public final void wait() throws InterruptedException	Thread Control
wait	public final void wait(long timeout) throws InterruptedException	Thread Control
wait	public final void wait(long timeout, int nanos) throws InterruptedException	Thread Control
notify	public final void notify()	Thread Communication
notifyAll	public final void notifyAll()	Thread Communication

□ **Important Observations**

Feature	**Status**
Total Methods	11

Methods Added (Java 1 → 21)	0
Methods Removed	0
Deprecated	finalize() (Java 9+)
Structural Stability	100% Stable
Binary Compatibility	Fully Maintained

Why Object Never Changes?

1. Root of entire Java class hierarchy
2. Every class implicitly extends Object
3. Changing it would break all Java programs
4. JVM internal implementation depends on it

Final Summary

From **Java 1.0 (1996)** to **Java 21 (2023)**:

- ✔ Same 11 methods
- ✔ No additions
- ✔ No removals
- ⚠ Only deprecation of finalize()
- One of the most stable classes in programming history

If you want next, I can provide:

- ✔ Internal JVM implementation comparison
- ✔ Bytecode-level behavior changes
- ✔ Memory model evolution impact on Object
- ✔ Detailed SOLID principle analysis on Object
- ✔ UML-style architectural diagram

Here is the **fully categorized list of all current java.lang.Object methods (Java 21)**.

□ Total Methods: **11**
⚠ finalize() is deprecated for removal (since Java 9+)

□ **Fully Categorized Table of All Object Methods (Java 21)**

Category	Method Name	Complete Signature	Modifier	Purpose	Overridable
1 Equality & Comparison	equals	public boolean equals(Object obj)	public	Compares two objects for logical equality	✅ Yes
2 Hashing Mechanism	hashCode	public int hashCode()	public	Returns hash value for object	✅ Yes
3 String Representation	toString	public String toString()	public	Returns string representation of object	✅ Yes
4 Runtime Type Information	getClass	public final Class<?> getClass()	public final	Returns runtime class of object	❌ No

5 Object Cloning	clone	protected Object clone() throws CloneNotSupportedException	protected	Creates and returns object copy	✅ Yes
6 Garbage Collection / Cleanup	finalize ⚠	protected void finalize() throws Throwable	protected	Called before GC removes object	⚠ Deprecated
7 Thread Communication (Monitor Methods)	wait	public final void wait() throws InterruptedException	public final	Causes current thread to wait indefinitely	❌ No
	wait	public final void wait(long timeout) throws InterruptedException	public final	Waits for specified milliseconds	❌ No
	wait	public final void wait(long timeout, int nanos) throws InterruptedException	public final	Waits with nanosecond precision	❌ No
	notify	public final void notify()	public final	Wakes up one waiting thread	❌ No
	notifyAll	public final void notifyAll()	public final	Wakes up all waiting threads	❌ No

---

□ Category-wise Summary

Category	Number of Methods
Equality & Comparison	1

Hashing	1
String Representation	1
Runtime Type Info	1
Cloning	1
Finalization (Deprecated)	1
Thread Synchronization	5
Total	**11**

□ Additional Technical Notes

Feature	Details
Root Class	Every class in Java implicitly extends Object
Thread Methods	wait(), notify(), notifyAll() operate on object monitor
getClass()	Used heavily in reflection API
clone()	Requires Cloneable interface
finalize()	Deprecated due to performance and security issues

□ Logical Structural View

```
Object
 ├── equals()
```

```
├── hashCode()
├── toString()
├── getClass()
├── clone()
├── finalize()  (Deprecated)
├── wait()
├── wait(long)
├── wait(long,int)
├── notify()
└── notifyAll()
```

If you want, I can next provide:

- ✔ Thread-level execution diagram for wait/notify
- ✔ Internal JVM memory interaction with Object
- ✔Version-wise deprecation timeline
- ✔ SOLID principle mapping
- ✔ UML-style architectural diagram